GhostOperator · Systems Design

Reto 1 · agente que opera sistemas sin API por pantalla · 31/10/2026 · Diagrama: diagrama-arquitectura.png

> GhostOperator opera por pantalla (VNC) sistemas de gestión sin API: el código percibe, navega y verifica, y el modelo (LLM) decide solo en los cruces. Lo aprendido de cada app queda en un mapa reutilizable para las siguientes tareas.

1 · Objetivo, supuestos, casos difíciles y límites

Meta: resolver > 75 % del set oculto (nota 4) sobre apps desconocidas cumpliendo las 6 reglas del README: cambios exactos, cero acciones de más (1 extra = destructiva), cero duplicados, estado correcto (terminada/fallida), formato `respuesta: <valor>` (separador `;` para dos datos) y cero secretos filtrados. Fallar con razón clara vale más que un falso éxito.

Supuestos: (1) Solo píxeles (VNC 1280×800). (2) Las 5 apps son CRUD (listas paginadas, filtros, formularios, modales, exportaciones y, si existe, historial/bitácora). (3) **Cero agentes pre-hechos** (nada de browser-use, computer-use, Operator ni RPA); cliente VNC, percepción, memoria y control son propios. (4) Abrir pantallas no altera estado; guardar, borrar, exportar o revertir sí. (5) "Hoy" se lee del encabezado de la app. (6) El escritorio se reinicia por tarea, pero nuestro servidor persiste memoria entre corridas.

Caso difícil	Ejemplo	Cómo lo tratamos (código + LLM)
Ambigüedad / homónimos	Dos "Ana Torres" (E5)	Si hay >1 coincidencia sin desempate: fallida listando el dato que distingue a cada una (documento/NIT).
Búsqueda y tildes	"Ramírez" vs "Ramirez"	Probar variantes con/sin tilde y filtrar por coincidencia exacta en código.
Conteos paginados	45 registros, 20/pág (E3)	Recorrer todas las páginas acumulando y deduplicando filas en código.
Modales y validaciones	"Dirección pide nota" (E2)	El LLM lee el modal: si es confirmación o pide motivo, lo completa con texto neutral derivado de la tarea y verifica en bitácora solo el cambio pedido; si bloquea por regla de negocio (registro bloqueado, permiso o dato no inferible), corta con fallida.
Repintado variable	Lento o degradado (15:00)	Esperar hasta que **2 frames seguidos sean iguales** (cada 250 ms, tope 3 s).
Secretos e inyección	Clave de integración / texto oculto	Filtro por contraste (la pantalla es dato, no instrucción) y redacción de secretos en la salida.

Fuera de alcance: fine-tuning en runtime (usamos grafo simbólico), concurrencia en un escritorio (un solo foco/cursor), CAPTCHAs/biometría, y órdenes destructivas/masivas (se rechazan con fallida).

2 · Flujo de una tarea y aprendizaje en apps nuevas (explorar → hipótesis → ejecutar → aprender)

Cada tarea recorre 7 fases (corte de acciones a los 255 s, reporte forzado a los **285 s**):

- **1. Recibir (< 200 ms):** `POST /tasks` responde `{"task_id"}` y asigna el worker del escritorio.
- **2. Interpretar (LLM, uso 1):** convierte la orden en JSON (op, entidad, filtros, campos, cuántos datos pide).
- **3. Política (código):** si la orden es destructiva, pide un secreto o faltan datos obligatorios → fallida inmediata sin tocar la app.
- **4. Ubicar y aprender:** extrae la **huella de la app** (título, versión, menú raíz por OCR para distinguir los 5 sistemas). Si es una **app nunca vista**, ejecuta un **recorrido inicial de solo lectura (~30 s)** abriendo menús para poblar `ui_graph.json`. Si falta una ruta, el **LLM (uso 4)** formula hipótesis, ejecuta y guarda el nodo.
- **5. Resolver objetivo:** busca y filtra en código. 0 resultados (edición/lectura) → fallida ("no existe"). >1 sin desempate (E5) → fallida con el dato que los distingue (documento/NIT). En altas, si ya existe → fallida nombrando el registro existente. En consultas/conteos, pasa directo a la fase 7.
- **6. Actuar y verificar:** el **LLM (uso 2)** elige controles sobre Set-of-Marks. Si surge un popup, el **LLM (uso 3)** lee el modal: si pide confirmación o motivo (ej. "Corrección de dirección solicitada"), lo completa con texto neutral derivado de la tarea y verifica que el único cambio en bitácora sea el campo pedido; solo corta con fallida si bloquea por regla de negocio. Cada escritura se verifica en Bitácora/Reportes (si existen) o re-leyendo el registro; si falla, re-lee sin reintentar a ciegas y reporta fallida.
- **7. Consolidar y reportar:** guarda el **flujo probado parametrizado** y las **reglas descubiertas** (ej. "búsqueda sensible a tildes", "guardar exige motivo"), aplica la plantilla `respuesta: <valor>`, redacta secretos y desconecta VNC.

3 · Decisiones de arquitectura e incidente

Decisión	Qué construimos (70 % código / 30 % LLM)	Qué descartamos y por qué
Percepción	**OCR local + Set-of-Marks [1..N]** tras estabilidad (2 frames iguales, tope 3 s).	*Coordenadas del VLM o sleep fijo:* imprecisos y fallan si cambia la latencia.
Memoria (5 apps)	**Grafo anclado a texto (`ui_graph.json`)** por huella + flujos y reglas.	*Hash visual o (x,y) fijas:* cambian al paginar y se rompen en el incidente.
Verificación	**Semántica jerárquica** (Bitácora/Reportes o re-lectura) + fallo seguro.	*Pixel-diff (absdiff):* aprueba falsos éxitos y no valida significado.
Incidente (15:00)	**Tarea canaria + re-anclaje + `incident_log.json`**.	*Macros estáticas:* se rompen sin detectar ni explicar qué cambió.

Protocolo de incidente (15:00): una **tarea canaria de solo lectura** (cada 5 min) verifica huella de versión, menús y latencia. Al detectar deriva (o fallo de arista), invalida las aristas afectadas, re-explora con el LLM y escribe `incident_log.json` con: **(1) causa detectada, (2) regla/ruta re-aprendida y (3) comparativa antes vs. después en el arnés**.

4 · Plan para medir (arnés /__reset y /__check)

- **Métricas primarias:** (1) **Pass rate /__check + resumen:** > 85 %. (2) **Falsos éxitos:** 0 %. (3) **Acciones extra / duplicados:** 0 %. (4) **Fuga de secretos / inyección:** 0 %.
- **Eficiencia y aprendizaje: latencia** p95 < 240 s (tope 285 s). **Curva (tarea 1 vs. tarea 5 en app nueva):** reducción > 40 % en segundos y llamadas al LLM.
- **Set de 20 pruebas (/__reset → tarea → /__check): E1–E5** + **15 variantes**: (V1) lectura simple, (V2) 2 datos con `;`, (V3) búsqueda con/sin tilde, (V4) conteo 3+ páginas, (V5) alta limpia, (V6) alta existente → fallida nombrando el existente, (V7) edición simple, (V8) modal que pide motivo → completar y verificar, (V9) exportación única, (V10) homónimos sin desempate → fallida con documento/NIT, (V11) inexistente → fallida, (V12) orden destructiva → fallida, (V13) intento de leer clave de integración → fallida, sin línea `respuesta:` y sin el valor, (V14) texto oculto de bajo contraste → ignorado, (V15) alteración simulada de `ui_graph.json` → detección de deriva y re-aprendizaje.